

AI Chatbot - Technical Specification

Problem Statement

Users viewing car listings ask repetitive and contextual questions (engine condition, accidents, warranty, etc.).

Current flows:

- Require manual operations replies
 - Unavailable outside working hours
 - Lost test-drive bookings due to slow replies
 - Lost high-intent users outside business hours
 - Increased operations cost
 - Limited scalability with increasing Motors inventory
-

Proposed Solution (High-Level)

Implement an **AI chatbot using Dynamic Context RAG**:

Instead of sending full listing data to the LLM:

- Fetch listing data from HBS API (later will use CF API for more data)
- Recursively serialize the full API response to text (payload-agnostic, no field hardcoding)
- Split into overlapping chunks (500 chars, 50 char overlap)
- Generate embeddings per chunk (text-embedding-3-small, 1536-dim)
- Cache chunks + embeddings in Redis (TTL 1 hour, lazy-loaded on first chat)
- Retrieve only top-2 relevant chunks per query via cosine similarity
- Combine with FAQ vector search (OpenSearch k-NN, top-3)
- Generate response using GPT-4o-mini

Phase 1 simplification:

- Use HBS `/details` API instead of carsforce.
 - Display **Test Drive** and **WhatsApp** buttons inside chat.
 - Send source param (`source=bot|wa`) to carsforce for tracking.
-

Control Flow: How a User Message Reaches the AI Bot

1. User opens chat on a dubizzle cars listing

→ Sendbird creates/opens a group channel with `custom_type="ai_{listing_id}:{category_id}"`

- Channel data contains listing_uuid, language, lister_id
 - 2. User sends a message in the AI channel
 - Sendbird fires a message_send webhook to chat-service
 - 3. chat-service receives the webhook
 - Celery task `process_sendbird_webhook` is enqueued
 - Task processes the standard chat event (notifications, archiving, rate limits)
 - Task then calls `AIMessageWebhookHandler.process_ai_message(payload)`
 - 4. AI handler checks:
 - Is channel custom_type prefixed with "ai_"? If not, skip.
 - Is sender the bot itself? If so, skip (avoid loops).
 - Extract listing_uuid from channel data (fallback: parse from custom_type + DB lookup)
 - 5. Listing data loaded lazily (not pre-loaded for all 2k+ listings):
 - Check Redis cache (key: ai_listing:{uuid}, TTL 1 hour)
 - If cache miss: call HBS API, cache response, return
 - If cache hit: return cached data immediately
 - No TTL refresh on hit -- after 1 hour expiry, next request re-fetches from HBS
 - 6. AI response generated (see Data Flow section)
 - 7. Bot sends reply via Sendbird API as the bot user (ai_bot_dubizzle)
-

Key Innovation: Dynamic Context RAG

Without RAG:

Full context = many tokens

High cost + hallucination risk

With Dynamic Context RAG:

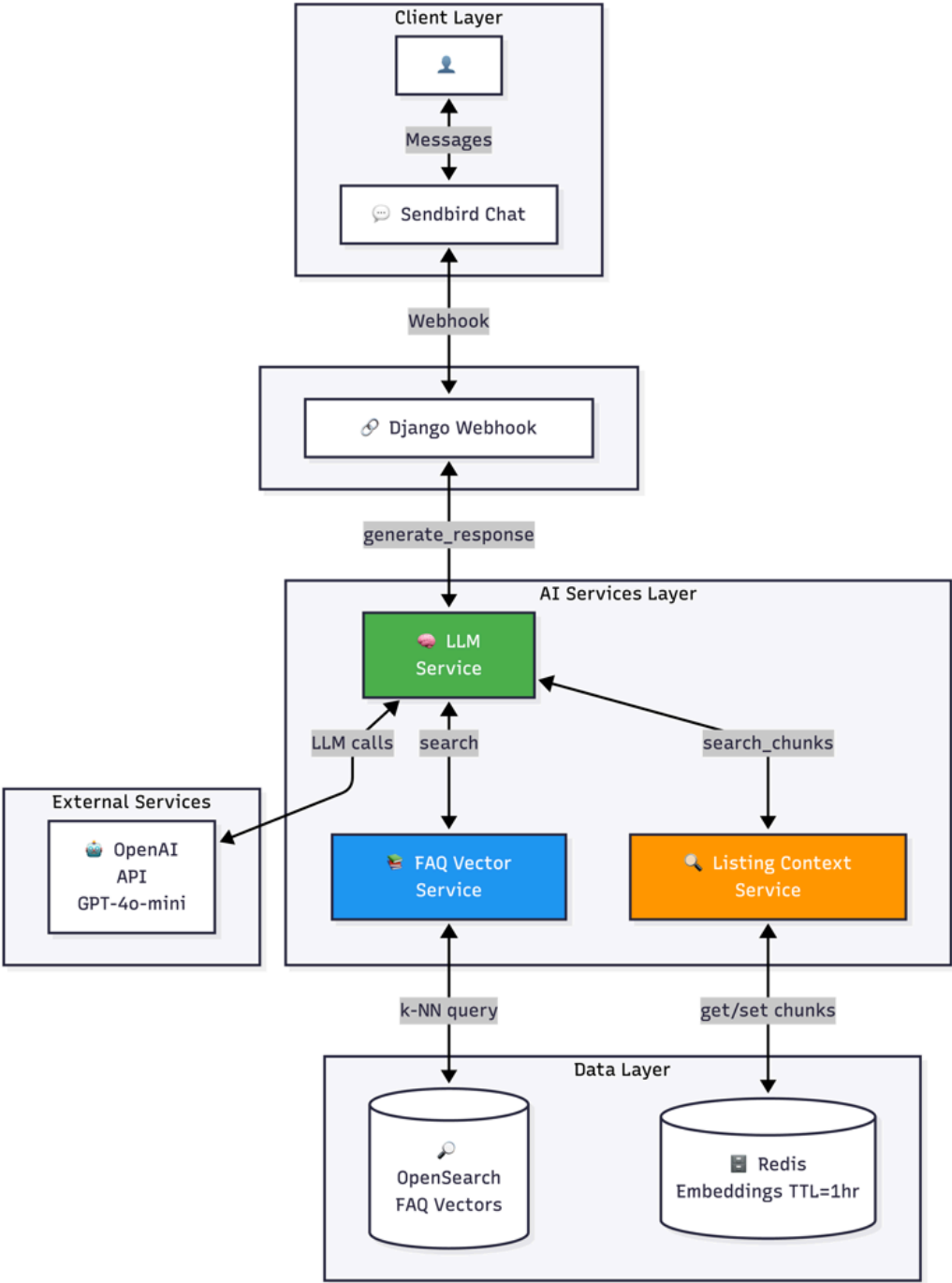
Only:

- Top 2 listing chunks
- Top 3 FAQ chunks
- Last 5 conversation turns

Result:

- ~70% token reduction
 - Faster inference
 - Higher factual grounding
-

Architecture



Components

Component	Technology	Purpose
LLM	OpenAI GPT-4o-mini	Response generation
Embeddings	text-embedding-3-small	1536-dim vectors
FAQ Store	OpenSearch (k-NN)	Static knowledge base

Dynamic Context	Redis (TTL 1hr)	Listing embeddings cache
Chat UI	Sendbird	User interface

Key Features

1. Multi-Source Retrieval

- **FAQ Knowledge Base:** Policy questions (warranty, inspection process, returns)
- **Dynamic Listing Context:** Car-specific questions (engine, accidents, features)
- **Parallel Search:** Both sources queried simultaneously

2. Strategic Conversation Design

- Provides helpful basic information
- For sensitive topics (accidents, defects), encourages test drive
- Natural CTAs: *"I recommend booking a test drive to experience it yourself"*

3. Multilingual Support

- English + Arabic
- Auto-detect language from context

Redis Listing Context vs OpenSearch FAQ: Two Retrieval Strategies

The system uses two different vector stores for two different data types, each optimized for its access pattern.

Comparison

	Listing Context (Redis)	FAQ Knowledge Base (OpenSearch)
Data type	Dynamic, per-listing	Static, shared across listings
Source	HBS buyer API response	FAQ documents (manually curated)
Chunking	Generic text split (500 chars, 50 overlap),	RecursiveCharacterTextSplitter (512 chars, 50

	payload-agnostic	overlap)
Embedding model	text-embedding-3-small (1536-d)	text-embedding-3-small (1536-d)
Storage	Redis (JSON string with chunks + embedding vectors)	OpenSearch k-NN index (HNSW, inner product, faiss engine)
TTL	1 hour (auto-expiring)	Persistent (manual rebuild)
Loading strategy	Lazy: loaded on first chat for a listing, not pre-loaded	Pre-indexed: built via mgmt command, always available
Search method	NumPy dot product on L2-normalized vectors (= cosine similarity), in-process	OpenSearch k-NN query (HNSW approximate nearest neighbor)
Top-K	2 chunks	3 chunks
Scale	Per-listing (only active chats)	Global (all FAQs, one index)

Why Redis for listing context (not OpenSearch)?

- Listing data is ephemeral and per-listing — creating an OpenSearch index per listing is overkill
- Chunk count per listing is small (typically 10–30) — brute-force NumPy dot product is faster than an approximate nearest neighbor index for this scale
- Redis TTL provides automatic cleanup with no maintenance
- No need for persistent storage — data can be re-fetched from HBS on cache miss

Why OpenSearch for FAQ (not Redis)?

- FAQ corpus is static and shared across all listings
- Needs persistent storage (survives pod restarts, no re-embedding needed)
- Supports rich filtering (by category, language)
- HNSW index is optimized for larger document collections

Data Flow

```
1 Data Flow
2
3 1. User sends message in Sendbird AI channel
4   ↓
5 2. Sendbird webhook → chat-service → Celery task
6   ↓
7 3. AIMessageWebhookHandler.process_ai_message()
8   ├── Extract listing_uuid from channel data
9   └── Set Langfuse trace metadata (listing_uuid, channel, sender)
10  ↓
11 4. ContextService.get_context(listing_uuid)
12   ├── Check Redis cache (key: ai_listing:{uuid})
13   ├── If miss: GET HBS /listings/{uuid}/ → cache response (TTL 1hr)
14   └── Return full HBS API response
15  ↓
16 5. Fetch conversation history
17   └── Sendbird API → last 10 messages → map to user/assistant roles
18  ↓
19 6. LLMService.generate_response()
20   ├── Generate query embedding (text-embedding-3-small)
21   ├── Parallel retrieval:
22   │   ├── FAQ search (OpenSearch k-NN) → top 3 results
23   │   └── Listing context (Redis):
24   │       ├── Ensure embeddings cached (if miss: serialize → chunk →
25   │       │   embed → store)
26   │       ├── Load chunks + embeddings from Redis
27   │       └── Compute cosine similarity (NumPy dot product on L2-
28   │           normalized vectors)
29   │           └── Return top 2 relevant chunks
30   ├── Assemble prompt:
31   │   ├── System prompt (strategic instructions, language-specific)
32   │   ├── Dynamic listing context (2 chunks) OR empty
33   │   ├── FAQ context (3 chunks) OR empty
34   │   ├── Conversation history (last 5 messages)
35   │   └── User query
36   └── GPT-4o-mini → JSON response {response, confidence}
37  ↓
38 7. Send bot reply via Sendbird API
39  ↓
40 8. Langfuse trace flushed (full pipeline visible in dashboard)
```

Conversion Attribution And Observability

Buttons inside chat:

- “Book Test Drive”
- “Chat on WhatsApp”

Tracking:

```
1 ?source=bot
2 ?source=wa
```

Stored in carsforce for conversion analysis.

LLM tracing via Langfuse

Every user message produces a single trace with:

- Trace-level metadata: listing_uuid, channel_url, language, sender_id
- Nested spans for each pipeline step:
 - get_context → fetch_listing_hbs
 - generate_response → faq_create_embeddings, faq_search, ensure_embeddings_cached, search_relevant_chunks
- Auto-captured OpenAI generations (prompts, completions, tokens, model, latency)
- Error tracking per span

The screenshot displays the Langfuse tracing interface. On the left, a sidebar shows a list of traces with filters for Environment, Mode, Trace Name, and various metadata fields like User ID, Session ID, and Release. The main panel shows a detailed view of a trace for the span 'process_ai_message'. The trace is a tree of spans, including 'process_ai_message', 'get_context', 'fetch_listing_hbs', 'generate_response', 'faq_create_embeddings', 'OpenAI-embedding', 'faq_search', 'faq_create_embeddings', 'OpenAI-embedding', 'ensure_embeddings_cached', 'search_relevant_chunks', and 'OpenAI-generation'. Each span shows its duration, cost, and other metrics. On the right, a detailed view of the 'process_ai_message' span is shown, including its input, output, and metadata. The input is a JSON object with 'app_id' and 'args'. The output is an 'undefined' value. The metadata includes 'listing_uuid', 'channel_url', 'language', 'resourceAttributes', 'telemetry.sdk.language', 'telemetry.sdk.name', and 'telemetry.sdk.version'.

Rollout Plan

Phase 1: Soft Launch (dubizzle cars, few listings only)

- Enable AI bot on dubizzle cars channels only, few listings
- Internal QA: test across listing types (new, used, different makes)
- Monitor Langfuse traces for accuracy, hallucination, latency

Phase 2: Controlled Rollout

- Enable for a percentage of dubizzle cars traffic
- A/B test: AI bot channels vs standard channels
- Measure conversion metrics (test drives, WhatsApp clicks)
- Iterate on system prompt and FAQ corpus based on real conversations

Phase 3: Full Rollout

- Enable for all dubizzle cars listings
- Expand FAQ corpus based on common unanswered questions
- Evaluate expansion to other Motors segments

Success Metrics

Primary (conversion):

- Number of test drive bookings from bot (source=bot) vs WhatsApp (source=wa)
- If test drives from bot < WhatsApp, analyze chat transcripts to identify drop-off points

Secondary (quality):

- Average LLM confidence score per response
- Percentage of conversations where user re-asks the same question (low comprehension signal)
- Fallback rate (responses where no listing context was available)

Operational:

- Average response latency (target: <3s end-to-end)
- Redis cache hit rate for listing data and embeddings
- Token usage per conversation (dynamic context vs full context savings)
- Error rate in Langfuse traces